



Portfolio Site Plan — ahmarkhhan.github.io (Backend/AWS/AI Engineer)



Objective: Ship a fast, credible personal site at `ahmarkhhan.github.io` (later `ahmarkhan.dev`) that converts three audiences — recruiters, freelance clients, and future customers — by leading with **proof and metrics**, not visual flash. This plan is opinionated and steelmanned: for each major call I argue the strongest case *against* my recommendation, then show why the recommendation still wins.

0. TL;DR — The recommendation in one screen

- **Stack:** Astro + Tailwind CSS + TypeScript, with one or two React "islands" for interactive AI/demo widgets. Deploy via GitHub Actions to GitHub Pages.
- **Domain:** Buy `ahmarkhan.dev` (~\$12/yr) and point it at GitHub Pages. Keep `ahmarkhhan.github.io` as the fallback.
- **Positioning (the whole game):** *"Symfony/PHP ERP & portal engineer — AWS-certified — now building agentic AI systems."* One sentence, repeated everywhere.
- **Core content:** 3 deep case studies (FM Packages ERP, Nigehban AI, a utility-bill automation tool) written as Problem → Approach → Challenges → **Results with numbers**.
- **The single biggest lever:** attach metrics to your work. Across every 2026 hiring guide, quantified impact is what separates portfolios that get interviews from those that don't — and it is also your documented #1 gap.

- **Timeline:** ~2 focused weekends to a strong v1; a live URL beats a perfect draft.
-

1. Market analysis — what actually matters in 2026

1.1 The uncomfortable baseline

A hiring reviewer of ~200 dev portfolios in the last year reports **90% make the same mistakes**: identical layouts, tutorial projects (todo/weather/calculator clones), and an "About Me" that says "passionate developer who loves solving problems." AI site-builders have made *pretty* the default, so pretty no longer differentiates.

Implication: The scarce commodity is not design — it is **evidence and narrative**. That is good news for you: a backend engineer with real production ERP systems has better raw material than most.

1.2 What recruiters and clients look for (converged signals across 2026 guides)

- **Projects before bio.** Visitors come for skills and work; surface projects first or immediately after a one-line intro.
- **Case studies, not screenshots.** For top 2–3 projects: the problem, the technical choices *and why*, what went wrong, and measurable results.
- **Honest, specific stack.** "Symfony (2.5 yrs, production)" beats a wall of 25 logos. Depth in one area + breadth around it is the sweet spot.
- **Speed and mobile.** Sub-2s load, Lighthouse 90+, flawless on phones — recruiters browse on mobile, and a slow site from an engineer is self-defeating.
- **Custom domain + working contact.** A real domain and a direct way to reach you read as professional.
- **AI literacy is now table stakes** — but a "ChatGPT wrapper" counts against you. What lands: RAG, embeddings/vector search, agentic orchestration, AI solving a real domain problem. Your Nigehban AI project is exactly this.

1.3 The backend-specific problem (and the opportunity)

Backend/ERP work is invisible: nobody sees an optimized query, a clean API, or a barcode-traceability module. The recurring advice for backend portfolios:

- Make the invisible visible: **architecture diagrams**, data-flow diagrams, ER models, before/after performance numbers.
- Provide **runnable proof**: a live API with public docs (Swagger/OpenAPI), a Postman collection, a short Loom demo video.
- Ship **public code with real READMEs**, not just a Gist — READMEs are judged as a communication skill.

1.4 What other engineers are shipping right now

- **Frameworks**: Astro dominates content/portfolio sites (ships ~0–9 KB JS vs hundreds of KB for a typical React/Next app; independent benchmarks put Astro 2–3× faster to load). Next.js is the default when people expect the site to grow into an app. Plain HTML/Hugo still used by minimalists.
- **Look**: minimalism, dark-mode-first, strong typography, tasteful motion; a smaller cohort does 3D/GSAP showpieces (high effort, mostly for frontend specialists).
- **Content trend**: "showcases" (tight, scannable, results-first) are replacing long academic case studies. Hiring managers scan; lead with impact.

2. Strategic positioning — your unfair advantage

Don't present as "a full-stack developer" (undifferentiated, competes with everyone). Present as a **domain specialist with an AI edge**:

Ahmar Khan — I build and modernize ERP & portal systems for real operations (packaging, healthcare regulation, public policy), deploy them on AWS end-to-end, and I'm now building agentic AI on top.

Why this wins:

- **Niche beats generalist.** The strongest portfolio examples went *deep* on one thing (iOS accessibility, ERP modernization) and stood out from the generic crowd.
- **You already own the niche.** Corrugated-manufacturing ERP from scratch, Symphony 2.8 → 4 modernization (PNC), multi-level approval workflows (OPM) — few competitors can claim this.
- **The AI layer is timely.** AWS ML Specialty + Nigehban AI (Spring Boot reasoning engine, multi-spectral ingestion, real-time HUDs) lets you ride the 2026 AI-required wave without being a "wrapper."

Three-audience framing (same site, one toggle or three cards on the landing page):

Audience	What they need to see	Primary CTA
Recruiters (jobs)	Stack proficiency, production impact, resume PDF, LinkedIn	Download résumé / Contact
Freelance clients	Outcomes, testimonials, "how I work," services	Book a call / Email
Future customers (product)	Domain authority, that you ship real systems	Read the ERP case study

3. Tech stack decision (steelmanned)

3.1 Recommendation: Astro + Tailwind + TypeScript, React islands only where needed

Option	Pros	Cons	Verdict
Astro + Tailwind (pick)	Native static output, ~0 JS by default, 100 Lighthouse, first-class SEO/OpenGraph, content collections for case studies/blog,	Newer ecosystem; you'd learn a new framework	✅ Best fit for a static, content-first, SEO-driven site

Option	Pros	Cons	Verdict
	drop in React/Vue/Svelte islands for demos, deploys to GitHub Pages with zero fuss		
Next.js (static export)	Huge ecosystem, React skills transfer, room to grow into a full app	Needs <code>output: "export"</code> , image- optimization workarounds, and <code>.nojekyll</code> /asset- prefix hacks on GitHub Pages; ships far more JS for a brochure site	Only if you <i>plan</i> to grow this into a web app
Plain HTML/CSS (+ Alpine)	Zero build, ultimate control, fastest possible	No components/content model; case studies + blog get painful to maintain	Fine for a one- pager; weak for a growing site
Template (RyanFitzgerald/devportfolio etc.)	Live in an hour	Looks like everyone else's; low signal	Use only as reference, not as-is

3.2 Steelman AGAINST Astro (the strongest counterargument)

"You're a backend/ERP engineer. Learning a frontend framework to build a brochure site is a distraction, and a hand-polished frontend sends a mixed signal about your specialty. Just use plain HTML or a template and spend your time on case studies and metrics."

This is a *serious* argument — your scarce resource is time, and content is where the ROI is. **Why Astro still wins:**

1. Astro is closer to HTML than React is; the learning cost is a few hours, not weeks.
2. Content collections make Problem → Result case studies and a future blog *cheaper* to maintain — it protects the thing that matters (content).

3. React islands let you embed the one thing that impresses for *your* profile: a **live AI demo** (e.g., a small RAG-over-your-projects chat, or a Nigehban-style live feed widget). A static template can't.
4. The build itself becomes a credibility artifact (100 Lighthouse, clean repo) without becoming a 3D-animation time sink you don't need.

Net: Astro is the minimal frontend investment that maximizes content leverage — it does *not* pull you toward frontend specialization.

3.3 Supporting choices

- **Styling:** Tailwind CSS (fast, consistent, dark-mode built in).
- **Content:** Markdown/MDX via Astro content collections for case studies + notes.
- **Analytics:** Plausible or Umami (lightweight, privacy-friendly) — know which projects get viewed.
- **Forms/contact:** mailto + a Formspree/Web3Forms endpoint (no backend needed on a static host).
- **AI island (differentiator):** a serverless function on Cloudflare Workers/Vercel free tier for any live AI demo, called from the static site (keeps GitHub Pages purely static).

4. Deployment plan — GitHub Pages specifics

- **Repo:** create `ahmarkhhan.github.io` (user site → served at the root domain, cleanest URL).
- **Build:** GitHub Actions workflow (`withastro/action`) builds and publishes on every push to `main` . No manual `gh-pages` juggling.
- **Static only:** GitHub Pages serves static files exclusively — Astro's default output fits natively (this is the pain point people hit with Next.js).
- **Custom domain:** add a `CNAME` file with `ahmarkhhan.dev` , set DNS (A/ALIAS to GitHub Pages IPs + CNAME for `www`), enable *Enforce HTTPS*.
- **Add** `.nojekyll` so GitHub doesn't run Jekyll over your build output.

- **Target:** Lighthouse ≥ 90 on Performance/Accessibility/SEO/Best-Practices before you share the link.
-

5. Site architecture (information design)

Recommended: multi-section single scroll + dedicated case-study pages.

1. **Hero** — name, the one-line positioning statement, primary CTAs (Résumé, Email, GitHub, LinkedIn). Availability badge ("Open to roles / freelance").
 2. **Selected Work (first!)** — 3 project cards, each with a one-line outcome + metric. Links to full case studies.
 3. **Case study pages** (</work/fm-packages-erp> , </work/nigehban-ai> , </work/bill-automation>).
 4. **About** — short, specific: domain focus, AWS certifications, how you work. Avoid clichés.
 5. **Tech stack** — grouped by proficiency with honest levels and years.
 6. **Achievements** — hackathon rank (10th/600), AWS Community Builder, certifications.
 7. **Writing/Notes** (optional, high-ROI later) — short technical posts feed SEO and personal brand.
 8. **Contact** — email, form, LinkedIn, GitHub, location (Islamabad, open to remote).
-

6. Content plan — what to REMOVE, ADD, and REFRAME

This maps your Career Master Record into a public site. The master record is a *history document*; the site is a *conversion document*. Ruthless editing is the job.

6.1 REMOVE / keep private

- **Personal data:** date of birth, student email, full home details — omit. Use a professional email only.

- **Section 9 "Open Items" and Section 10 "Readiness Gap Analysis"** — internal planning, never public.
- **Unverified/half-projects** ("few other portals") — omit until named and one-lined.
- **Self-deprecating notes** (e.g., "less confident speaking", "metrics not measured") — never surface weaknesses on the site.
- **Long prose bios** — compress hard; the site rewards scannability.
- **Raw list of 5+ Cogilent projects with equal weight** — demote to a compact "Experience" strip; promote only the 2–3 that become case studies.

6.2 ADD (gaps to fill before/around launch)

- **A professional email + phone** on the résumé (documented as missing).
- **Real GitHub repos with READMEs** — replace the lone Gist. Push: an ERP module extract, the bill scraper/emailer, and a Nigehban AI component. READMEs are graded.
- **Quantified metrics** on every project — the highest-leverage add (see §7).
- **Architecture/data-flow diagrams** for the ERP and Nigehban AI — makes backend work visible.
- **A live or recorded demo** — Loom walkthrough or an embedded AI island.
- **Résumé PDF** (1–2 pages, ATS-friendly, derived from the master record).
- **LinkedIn profile** linked prominently — effectively mandatory for recruiter inbound.
- **1–2 testimonials** (even short) from FM Packages / past clients — social proof for the freelance audience.
- **OpenGraph/Twitter cards + favicon** so shared links look credible.
- **llms.txt / clean semantic HTML** — as AI agents increasingly summarize people, machine-readable structure helps you get represented accurately. Cheap, forward-looking edge.

6.3 REFRAME (same facts, sharper framing)

Master record says	Site should say
"Built ERP from scratch for a corrugated packaging factory"	"Designed & shipped a full corrugated-manufacturing ERP — from raw-material intake to barcode-traced production, finished-goods, billing & ledgers — used daily by floor operators, accounts, and leadership."
"barcode scanning of reels/raw material"	"Barcode traceability linking each reel to the job that consumed it — enabling stock governance and production planning that didn't exist before."
"portal modernization from Symphony 2.8 to 4" (PNC)	"Led a legacy Symphony 2.8 → 4 modernization for a national nursing regulator's licensing workflow (application → multi-level approval → SMS delivery)."
"AWS deployment end-to-end"	"Owned AWS delivery from EC2 provisioning through CI/CD to production, plus SQL backup/recovery for live systems."
"Nigehban AI (formerly AegisNet)"	"Nigehban AI — an autonomous national crisis-intelligence platform: event-driven Spring Boot reasoning engine, keyless multi-spectral ingestion (Meteo, GDELT, social), and real-time tactical HUDs (Angular + Kotlin). Regional top-10 of 600 teams."

7. The metrics problem — solve it, because it's your #1 gap

Your own gap analysis names quantified results as the single biggest weakness, and every 2026 guide agrees results-with-numbers is what earns interviews. You don't need perfect telemetry — reconstruct credible, defensible numbers:

- **Scale:** # of machines/operators using the ERP, # of clients tracked, records/rows in production, # of modules shipped.
- **Reliability/ops:** deployment cadence, uptime, backup/recovery events handled.
- **Before/after:** "stock selection was manual → barcode-traced," "no production planning → planned via ERP." Even directional "from 0 to X" statements are strong.

- **Team/scope:** team size, your role, duration, systems integrated.
- **AI project:** # of data feeds ingested, latency of the reasoning loop, hackathon rank (10th/600 — use it).

Rule: **every project card gets at least one number.** If you truly can't measure, state the qualitative before → after transformation instead — that still beats a feature list.

8. Case study template (use for all 3)

Title: <Impact-oriented, e.g. "Barcode-traced ERP for a corrupted factory">

One-liner **outcome** (with a number)

Context — company, industry, your role, team size, timeframe

The problem — what was broken / missing, and why it mattered

The approach — architecture + key tech choices **AND** why you chose them

Challenges — what went wrong, trade-offs, how you handled them

Results — metrics / before→after (this is the payoff)

Artifacts — diagram, screenshots, repo link, demo video

Stack — honest list for this project

Keep each scannable (PCR — Problem → Change → Retrospective — for the fast read; expandable detail below for those who want depth).

9. Extra-high plays (differentiation moves most engineers skip)

1. **Live "Ask my portfolio" AI island** — a small RAG chat grounded in your case studies. On-brand (AWS ML + AI engineer), memorable, and proves you build

real AI, not wrappers. Serverless call from the static site.

2. **Public API demo** — deploy one small read-only API (a slice of an ERP concept) with OpenAPI/Swagger docs + Postman collection. Turns invisible backend skill into something a reviewer can click.
 3. **Architecture diagrams as hero images** — most devs show UI screenshots; you show systems. Instantly signals seniority for backend roles.
 4. **A short technical write-up** ("How I modernized a Symfony 2.8 monolith to 4" or "Barcode traceability in a factory ERP") — SEO + authority, and reusable as LinkedIn content.
 5. **Freelance-mode services block** — packaged offers ("Symfony ERP modernization," "AWS deployment & CI/CD") with a clear CTA, so the same site serves the business path.
 6. **Machine-readability** — semantic HTML + `llms.txt` so AI assistants describe you accurately when someone asks about you.
-

10. Risks & honest counter-cases

- **Over-engineering the site instead of the content.** Biggest risk. Guardrail: content and metrics first; visuals second; a live imperfect v1 beats a polished draft.
 - **"Is a portfolio even worth it in 2026?"** For senior FAANG-track roles, recruiters may screen on resume alone. But for **freelance clients, remote/international roles, and your business path**, a credible site is a conversion and trust asset — and all three are on your table. So: worth it, weighted toward the freelance/business audiences.
 - **AI feature scope creep.** Keep the AI island tiny and reliable; a broken demo is worse than none.
 - **Authenticity vs. inflation.** Reframe, don't fabricate. Defensible numbers only — you may be asked about them in interviews.
-

11. Execution roadmap

Phase	Work	Output
0 — Prep (½ day)	Pro email; buy <code>ahmarkhan.dev</code> ; write the one-line positioning; reconstruct metrics	Positioning + metrics doc
1 — Content (weekend 1)	Write 3 case studies + about + stack; draft diagrams; record 1 demo	All copy final in Markdown
2 — Build (weekend 2)	Astro + Tailwind scaffold; content collections; responsive + dark mode; SEO/OG	Local site, Lighthouse 90+
3 — Deploy	Repo <code>ahmarkhhan.github.io</code> ; Actions build; custom domain + HTTPS; analytics	Live at your domain
4 — Proof	Real GitHub repos + READMEs; résumé PDF; link LinkedIn; collect 1–2 testimonials	Full credibility stack
5 — Compound (ongoing)	Add AI island; publish 1 write-up/month; iterate on analytics	Growing authority + SEO

12. Definition of done (v1)

- ☐ Live at `ahmarkhan.dev` (HTTPS), fallback `ahmarkhhan.github.io`
- ☐ One-line positioning in the hero; projects visible above the fold
- ☐ 3 case studies, each with ≥ 1 metric and Problem → Result structure
- ☐ At least one architecture diagram and one live/recorded demo
- ☐ Honest, grouped tech stack
- ☐ Résumé PDF + LinkedIn + real GitHub repos linked
- ☐ Lighthouse ≥ 90 across all categories; flawless on mobile
- ☐ Analytics installed; OG/Twitter cards + favicon set



Key sources: [Portfolio that gets hired 2026 \(DEV\)](#) · [Astro vs Next.js 2026 benchmarks](#) · [Astro dev portfolio template \(100 Lighthouse\)](#) · [Backend portfolio examples \(make invisible work visible\)](#) · [What recruiters look for \(9 tips from 40 reviews\)](#) · [Case study structure 2026 \(PCR\)](#) · [Deploy Next.js to GitHub Pages \(static-only constraint\)](#)

</> [Website Requirements & Design Spec v1 — ahmarkhan.dev](#)